

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1
«Розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни: «Обчислювальні методи»

Виконала: студентка групи AI-241
Карвацька Єлизавета
Олегівна

Перевірив: ас. Баранов О.М

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Виконано запити до двох моделей через ендпойнт `/api/generate`. Спочатку було виконано просте фактичне питання, потім — задачу на міркування у 5 запусках для кожної моделі. Перед серією вимірів було виконано прогрівальний запит. Для порівняння використовувалися моделі `qwen3:4b` та `llama3.2:3b`

Таблиця 1 – Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ
eval_count	190	5
Час, с	24,41	0,56
Токенів/с	7,84	22,39

У наведеному запуску обидві моделі дали правильну відповідь «Київ». Для `qwen3:4b` значення `eval_count` становило 190 токенів, загальний час виконання — 24,41 с, а швидкість генерації — 7,84 ток./с. Для `llama3.2:3b` отримано 5 токенів, час виконання — 0,56 с та швидкість генерації — 22,39 ток./с. Таким чином, у цьому експерименті `llama3.2:3b` виконала просте фактичне завдання значно швидше.

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Ручка і зошит разом коштують 110 грн. Ручка на 100 грн дешевша за зошит. Скільки коштує ручка?»

Показник	qwen3:4b	llama3.2:3b
----------	----------	-------------

Правильних із 5	5 / 5	0 / 5
eval_count	1360,6	177,8
Час, с	192,27	11,93

Міркувальна модель qwen3:4b у всіх п'яти запусках отримала правильну відповідь — 5 грн. Середнє значення eval_count становило 1360,6 токена, а середній час виконання — 192,27 с. У відповідях qwen3:4b також спостерігалось поле thinking, що свідчить про використання додаткового процесу міркування перед формуванням кінцевої відповіді. Інструктивна модель llama3.2:3b у жодному з п'яти запусків не отримала правильну відповідь. Модель повертала різні неправильні результати: «10 грн», «110 грн», «no solution», «0 грн» та «100 грн».

Відповіді на питання до завдання 1

1. Куди пішли додаткові токени в міркувальній моделі? Відповідь:

Додаткові токени міркувальній моделі витрачаються на внутрішній процес аналізу та послідовного розв'язання задачі перед формуванням кінцевої відповіді. Це видно з експерименту: під час задачі на міркування qwen3:4b у середньому згенерувала 1360,6 токена, тоді як llama3.2:3b — лише 177,8 токена.

2. На яких типах задач міркувальні моделі дають перевагу?

Відповідь: Міркувальні моделі доцільно застосовувати для задач, де необхідне послідовне виконання декількох кроків: математичних і логічних задач, аналізу, планування та інших задач, що потребують багатокрокового опрацювання інформації. У проведеному експерименті qwen3:4b правильно розв'язала задачу про ручку та зошит у всіх п'яти запусках, тоді як llama3.2:3b не отримала правильної відповіді в жодному із запусків.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної? Відповідь: Інструктивна модель орієнтована насамперед на виконання інструкцій користувача та формування безпосередньої відповіді. Міркувальна модель додатково оптимізується для виконання багатокрокового міркування, що може збільшувати кількість згенерованих токенів та час отримання відповіді. У практичному експерименті це проявилось у значно більшому eval_count та часу виконання qwen3:4b під час задачі на міркування.

4. Чи означає більша кількість токенів, що модель «розумніша»? Відповідь: Ні, сама кількість згенерованих токенів не є показником якості моделі. У цьому експерименті більша кількість токенів у qwen3:4b супроводжувалася правильним виконанням задачі на міркування, але це не означає, що кількість токенів сама по собі визначає якість моделі. Якість необхідно оцінювати за конкретним завданням, правильністю результату, стабільністю та іншими показниками.

5. Як пояснити різницю у швидкості генерації між моделями? Відповідь: Різниця у швидкості генерації пов'язана з архітектурою та характеристиками моделей, а також з обсягом обчислень, необхідних для формування відповіді. У фактичному запиті llama3.2:3b показала швидкість 22,39 ток./с проти 7,84 ток./с у qwen3:4b. На задачі міркування різниця була ще більшою за загальним часом виконання: середній час для qwen3:4b становив 192,27 с проти 11,93 с для llama3.2:3b.

Висновок за завданням 1

Експеримент показав, що швидкість генерації та загальний час відповіді є різними показниками.

На простому фактичному питанні обидві моделі надали правильну відповідь. При цьому llama3.2:3b виконала запит за 0,56 с, тоді як qwen3:4b — за 24,41 с. Швидкість генерації становила відповідно 22,39 та 7,84

ток./с. На задачі про ручку та зошит ситуація відрізнялася. qwen3:4b правильно розв'язала задачу у всіх п'яти запусках, використовуючи в середньому 1360,6 токена та 192,27 с на виконання. llama3.2:3b була значно швидшою — у середньому 11,93 с, але не дала жодної правильної відповіді з п'яти. Для серії з 8 послідовних викликів, якщо орієнтуватися на середній час задачі на міркування, сумарний час становитиме приблизно 1538,16 с для qwen3:4b проти 95,44 с для llama3.2:3b, тобто близько 25 хв 38 с проти 1 хв 35 с. Це показує суттєву різницю в часових витратах при використанні міркувальної моделі. Практичний висновок: тип моделі необхідно обирати відповідно до типу задачі. Для простих фактичних запитів важливими можуть бути швидкість та низька затримка, тоді як для задач, що потребують послідовного міркування, додаткові обчислювальні витрати можуть забезпечувати правильніший результат.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

За методичкою потрібно було порівняти два system-повідомлення для моделей llama3.2:3b та qwen3:4b: «Ти лаконічний технічний асистент.» та «Відповідай лише українською, максимум двома реченнями.»

Вплив system-повідомлення

Модель	System	Довжина символів	Мова	Речень	Інструкцію виконано ?
llama3.2:3b	лаконічний асистент	286	українська	4	Ні
llama3.2:3b	українською 2 речення	154	українська	2	Так

qwen3:4b	лаконічний асистент	412	українська	5	Ні
qwen3:4b	українською 2 речення	167	українська	2	Так

Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	34	82	116
llama3.2:3b	українською 2 речення	43	48	91
qwen3:4b	лаконічний асистент	39	286	325
qwen3:4b	українською 2 речення	48	241	289

Відповіді на питання до завдання 2

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Відповідь: Обидві моделі точно дотрималися обмеження «максимум двома реченнями»: llama3.2:3b та qwen3:4b сформуvalи по 2 речення після відповідного system-повідомлення. Тому за цим критерієм переваги однієї моделі над іншою не виявлено.

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?

Відповідь: Ні. За результатами таблиці всі чотири відповіді були українською, тому випадків переходу на іншу мову попри пряму вказівку не зафіксовано.

3. Як змінився `prompt_tokens` при зміні `system`-повідомлення і чому?

Відповідь: Зміна `system`-повідомлення змінює вхідний контекст, тому `prompt_tokens` має змінитися відповідно до кількості токенів нового повідомлення. При переході на `system`-повідомлення з додатковою вимогою щодо української мови та максимальної кількості речень `prompt_tokens` збільшилися на 9 токенів для обох моделей: у `Llama3.2:3b` з 34 до 43, а у `qwen3:4b` з 39 до 48. Це пояснюється збільшенням обсягу системної інструкції.

4. Чому в `qwen3:4b` `completion_tokens` може бути значно більше, ніж у `Llama3.2:3b` навіть при однаковій довжині видимої відповіді?

Відповідь: Міркувальна модель може витрачати додаткові токени на внутрішній процес міркування, які не обов'язково відображаються у короткій фінальній відповіді. Це узгоджується з результатами завдання 1, де `qwen3:4b` генерувала значно більше токенів.

Висновок за завданням 2

`System`-повідомлення використовується для задання мови, стилю та формату відповіді, але саме по собі не гарантує абсолютного виконання інструкції.

Завдання 3. Підключення хмарної моделі

Методичка передбачає повторення запиту з локальною та хмарною моделями через OpenAI-сумісний API. Для порівняння використано локальну модель `Llama3.2:3b` та хмарну модель `gpt-4o-mini`.

Таблиця 3 – Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
----------	----------	--------

Назва моделі	llama3.2:3b	gpt-4o-mini
Час відповіді, с	11,84	2,31
completion_tokens	87	76
Довжина відповіді, символів	341	386
Мова відповіді	українська	українська
Фактичних помилок	1	0

Відповіді на питання до завдання 3

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Відповідь: : Для переходу з локальної моделі на хмарну було змінено приблизно 3 рядки коду — параметри `base_url`, `api_key` та `model`. Основна логіка формування та надсилання запиту залишилася без змін.

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Відповідь: Хмарна модель дала повнішу й точнішу відповідь. Наприклад, у поясненні різниці між HTTP та HTTPS вона правильно вказала на шифрування трафіку через TLS, тоді як локальна модель спростила пояснення і допустила неточність.

3. Чи були в локальної моделі фактичні помилки або проблеми з українською мовою?

Відповідь: Так. У тесті якості llama3.2:3b дала неправильну відповідь у математичному завданні, а також у звіті зафіксовано мовні та змістові неточності в окремих завданнях.

4. Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет?

Відповідь: Це можливо завдяки значно потужнішому серверному обладнанню та апаратному прискоренню на стороні провайдера. Час передачі даних мережею може бути меншим за час локального обчислення великої моделі.

Висновок за завданням 3

OpenAI-сумісний API спрощує заміну провайдера, оскільки основна логіка надсилання запиту може залишатися однаковою. Вибір між локальною та хмарною моделлю має враховувати якість, швидкість, вартість, конфіденційність і залежність від інтернету.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Для порівняння було проведено benchmark для короткого, середнього та довгого запитів. Оцінювалися TTFT — час до першого токена, загальний час виконання та кількість символів, згенерованих за секунду.

Таблиця 4 – Результати benchmark

Модель	Запит	TTFT, с	Загальний час, с	Символів/с
llama3.2:3b	short	2,170	2,869	9,1
llama3.2:3b	medium	2,179	7,172	42,8
llama3.2:3b	long	2,174	40,124	41,8

qwen3:4b	short	35,146	49,448	5,6
qwen3:4b	medium	219,263	231,498	1,2
qwen3:4b	long	251,669	308,022	4,4

У llama3.2:3b TTFT у всіх трьох тестах залишався приблизно на рівні 2,17 с. Для qwen3:4b TTFT був значно більшим та зростав зі збільшенням обсягу тестового запиту: від 35,146 с для короткого запиту до 251,669 с для довгого. Загальний час виконання також був більшим у qwen3:4b.

Відповіді на питання до завдання 4

1. У якій моделі кращий TTFT і чому саме в неї?

Відповідь: Кращий TTFT у llama3.2:3b: приблизно 2,17 с у всіх трьох тестах проти 35,146–251,669 с у qwen3:4b. У проведеному експерименті локальна модель швидше починала генерувати відповідь.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — поясніть.

Відповідь: Не обов'язково. TTFT показує лише затримку до першого токена, тоді як символів/с характеризує швидкість подальшої генерації; це різні етапи роботи моделі.

3. Як змінилося співвідношення моделей llama3.2:3b та qwen3:4b при переході від короткого промпу до довгого?

Відповідь: При переході від короткого до довгого запиту перевага llama3.2:3b за TTFT збільшилася. Для короткого запиту різниця становила приблизно 33 с (35,146 проти 2,170 с), а для довгого — приблизно 249,5 с (251,669 проти 2,174 с). Отже, зі збільшенням запиту різниця між моделями за TTFT стала значно більшою.

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Відповідь: Для надійного порівняння одного запуску недостатньо, оскільки на вимірювання можуть впливати навантаження системи,

кешування та інші випадкові фактори. Методично доцільніше використовувати кілька запусків і порівнювати їхні типові значення.

5. Чому холодний старт повільніший за теплий?

Відповідь: Під час холодного старту модель та необхідні дані можуть завантажуватися в пам'ять, тому перший запит має додаткові витрати. Після прогріву наступні запити можуть виконуватися швидше.

Висновок за завданням 4

У проведеному benchmark llama3.2:3b показала менший TTFT і менший загальний час виконання в усіх наведених тестах. Для інтерактивного чат-бота важливим є TTFT, а для масової генерації великих обсягів тексту — швидкість генерації та загальний час.

Завдання 5. Оцінювання якості роботи моделей

Для перевірки якості моделей було виконано шість різнотипних завдань: фактологічне питання, формування JSON, стиснення тексту, написання Python-функції, математична задача та пояснення предметної області.

Таблиця 5 – Результати тестування якості

Завдання	llama3.2:3b	qwen3:4b
Task 1	Правильно: 1960	Неправильно: 1984
Task 2	Невалідний JSON	Валідний JSON, але значення неточні
Task 3	Числові значення збережено, є мовні помилки	Числові значення збережено

Task 4	Коректна функція, але є додатковий текст	Коректна функція, але є додатковий текст
Task 5	Неправильно: 1 рейс	Request timed out
Task 6	Відповідь отримано, але є змістові та мовні помилки	Request timed out

У першому завданні llama3.2:3b надала правильний результат — 1960, тоді як qwen3:4b повернула 1984. У другому завданні llama3.2:3b сформувала невалідний JSON, а qwen3:4b — синтаксично валідний JSON, але з неточними значеннями. У третьому завданні обидві моделі зберегли основні числові значення, але в llama3.2:3b спостерігалися мовні помилки. У четвертому обидві правильно реалізували функцію підсумовування weight, але додали пояснення та приклад. У п'ятому завданні правильна маса вантажу становить $45 \times 8 + 75 \times 12 = 1260$ кг. Оскільки вантажівка може перевозити максимум 1200 кг, необхідно виконати 2 рейси. llama3.2:3b дала неправильну відповідь — 1 рейс, а qwen3:4b завершити запит не змогла через тайм-аут. У шостому завданні llama3.2:3b сформувала відповідь із змістовими та мовними неточностями, а qwen3:4b не завершила запит через тайм-аут.

Завдання	llama3.2:3b	qwen3:4b
Task 1	1/2	0/2
Task 2	0/2	1/2

Task 3	1/2	2/2
Task 4	2/2	2/2
Task 5	0/2	0/2
Task 6	1/2	1/2
Разом /12	5/12	6/12

Відповіді на питання до завдання 5

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Відповідь: Серед локальних моделей більше балів набрала qwen3:4b — 6/12 проти 5/12 у llama3.2:3b. Основні відмінності виникли у фактологічному, форматному та логічному завданнях.

2. На яких завданнях оцінки двох прогонів можуть розійтися? Що це говорить про надійність моделі?

Відповідь: Оцінки можуть відрізнятися насамперед у завданнях із вільною генерацією, міркуванням та формуванням структурованих відповідей. Це показує, що один запуск не завжди достатній для надійного оцінювання моделі.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Відповідь: Так, qwen3:4b виконала вимогу краще: вона сформувала синтаксично валідний JSON без зайвого Markdown-оформлення.

llama3.2:3b не виконала вимогу повністю, оскільки сформувала невалідний JSON.

4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

Відповідь: Так. У результатах зафіксовано мовні помилки llama3.2:3b у завданні на стиснення та змістові й мовні неточності у шостому завданні.

Показовий приклад провалу

Запит: «45 ящиків по 8 кг і 75 ящиків по 12 кг. Вантажівка перевозить максимум 1200 кг.

Скільки рейсів потрібно?»

Отримана відповідь: llama3.2:3b — «1 рейс».

Правильна відповідь: $45 \times 8 + 75 \times 12 = 1260$ кг; $1260 / 1200 = 1,05$, отже потрібно 2 рейси.

Причина: модель неправильно оцінила багатокрокову арифметичну задачу та не врахувала обмеження максимальної маси одного рейсу.

Висновок за завданням 5

Тест якості показав, що отримання відповіді від моделі ще не означає її фактичної правильності. Для використання результатів у програмних системах необхідна додаткова перевірка.

Завдання 6. Керування випадковістю: temperature і top_p

Було досліджено вплив параметрів семплінгу на варіативність відповідей моделі llama3.2:3b. Окремо перевірено відтворюваність результату при temperature = 0,0.

Таблиця 6 – Вплив параметра temperature

Temperature	Кількість запусків	Унікальних відповідей
0,0	5	1
0,7	5	5
1,5	5	5

При $temperature = 0,0$ всі п'ять відповідей були однаковими. При $temperature = 0,7$ та $temperature = 1,5$ кожен із п'яти запусків дав різну відповідь. Отже, збільшення $temperature$ призвело до збільшення варіативності генерації.

Таблиця 7 – Вплив параметра top_p

top_p	Кількість запусків	Унікальних відповідей
0,1	5	1
0,5	5	5
1,0	5	5

При $top_p = 0,1$ всі п'ять відповідей були однаковими. При $top_p = 0,5$ та $top_p = 1,0$ було отримано по п'ять різних відповідей. Таким чином, зміна параметра top_p також впливала на варіативність результатів.

Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
$temperature$	0,0
Унікальних відповідей	1
Усі відповіді однакові	Так

Відповіді на питання до завдання 6

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Відповідь: Для агента, який має повертати JSON, доцільно використовувати низьку температуру, наприклад 0,0–0,2. Це зменшує

випадковість і полегшує отримання стабільного структурованого результату.

2. Яке — генератору варіантів заголовків?

Відповідь: Для генерації творчих варіантів можна використати вищу температуру, наприклад 0,7–1,0. Вона збільшує різноманітність формулювань.

3. Чим вплив `top_p` відрізняється від впливу `temperature` за характером, а не за силою?

Відповідь: `Temperature` змінює форму всього розподілу ймовірностей токенів, роблячи його більш або менш концентрованим. `top_p` обмежує вибір множиною найімовірніших токенів, сумарна ймовірність яких досягає заданого порога.

4. Чи означає `temperature=0` повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

Відповідь: Ні, `temperature=0` не є математичною гарантією абсолютної відтворюваності в усіх системах. У проведеному експерименті всі 10 запусків дали однакову відповідь.

5. Наведіть найбезглуздішу відповідь, яку модель видала при `temperature=1,5`, і поясніть, чому вона взагалі могла з'явитися.

Відповідь: Найбільш невдалою була відповідь, у якій модель відхилилася від початкового запиту, додала зайві твердження та втратила логічний зв'язок із поставленим завданням. Це могло виникнути через високе значення `temperature = 1,5`, яке збільшує випадковість вибору наступних токенів і підвищує ймовірність нетипових та менш узгоджених відповідей.

Питання до чат-бота

1. Що саме математично робить параметр `temperature` із розподілом ймовірностей наступного токена?

Відповідь: Перед вибором наступного токена `temperature` масштабує `logits`.

За вищої температури розподіл стає пласкішим, а за нижчої — більш концентрованим навколо найімовірніших токенів.

2. Чим `top_p` відрізняється від `temperature` — обидва впливають на різноманітність, але за яким механізмом?

Відповідь: `Temperature` змінює ймовірності всіх кандидатів, а `top_p` відкидає менш імовірні токени й залишає найменшу множину кандидатів із сумарною ймовірністю не меншою за заданий поріг.

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

Відповідь: Зазвичай зручніше спочатку налаштувати один параметр, щоб розуміти його вплив. Одночасна зміна обох може бути потрібна для тонкого налаштування, але ускладнює інтерпретацію результатів.

4. Чому навіть при `temperature=0` мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

Відповідь: На результат можуть впливати недетермінізм обчислень із плаваючою комою, паралелізація, особливості реалізації семплера та інші компоненти системи. У конкретному експерименті, однак, усі 10 запусків були однаковими.

5. Які значення `temperature` використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Відповідь: Для структурованих даних зазвичай використовують низькі значення, приблизно 0,0–0,2. Для творчої генерації часто використовують вищі значення, приблизно 0,7–1,0 або більше залежно від задачі.

Висновок за завданням 6

У проведеному експерименті низькі значення `temperature` та `top_p` давали однакові відповіді, а збільшення параметрів підвищувало варіативність.

При `temperature = 0,0` у десяти повторних запусках було отримано одну

унікальну відповідь, тому в межах цієї конфігурації результат виявився відтворюваним.

Загальні висновки

1) Локальні мовні моделі можуть використовувати єдиний програмний інтерфейс для взаємодії з програмою. У роботі обидві моделі запускалися через API, що дозволило використовувати однаковий підхід до надсилання запитів та отримання результатів.

2) Затримка відповіді та швидкість генерації є різними показниками. У проведеному експерименті llama3.2:3b мала значно менший TTFT і загальний час виконання порівняно з qwen3:4b.

3) Режим міркування є інструментом із певною областю застосування. У проведеному експерименті qwen3:4b використала значно більше токенів і часу, але правильно розв'язала задачу на міркування у всіх п'яти запусках. Для простого фактичного питання така кількість додаткових обчислень не була необхідною.

4) Більша кількість токенів сама по собі не означає вищу якість моделі. Якість необхідно оцінювати за правильністю відповіді та відповідністю конкретному завданню.

5) Промпт не гарантує повного контролю над форматом і змістом відповіді мовної моделі. Під час тестування одна модель сформувала невалідний JSON, а інша — синтаксично правильний JSON із неточними значеннями. Тому в програмних системах необхідно використовувати додаткову валідацію результатів.

6) Параметри семплінгу впливають на варіативність відповідей. У проведеному експерименті низькі значення temperature та top_p давали однакові відповіді, тоді як збільшення цих параметрів призводило до появи різних результатів.

7) Повторні запуски є важливими для оцінювання мовних моделей. П'ять повторів задачі на міркування продемонстрували різну поведінку моделей, а окремий десятиразовий тест показав повну повторюваність результату в конкретній конфігурації.

8) Проведене дослідження показало, що вибір мовної моделі повинен залежати від типу задачі. При оцінюванні необхідно враховувати не тільки правильність відповіді, але й час виконання, кількість токенів, TTFT, варіативність та здатність дотримуватися заданого формату.

9) За результатами лабораторної роботи було отримано практичний досвід розгортання локальних мовних моделей, роботи з API, автоматичного збору метрик та порівняльного тестування моделей на різних типах задач.